PDTN Phase B Offline Notes

This file is meant to be useful during the contest, not descriptive about the contest.

Keep only what helps you solve problems fast:

- no normal internet
- local docs may exist, but search may be weak
- code templates matter more than theory paragraphs
- baseline first, then improvements

1. Fast Contest Strategy

For each problem, identify these immediately:

- task type: regression / classification / clustering / retrieval / optimization
- input type: tabular / text / embeddings / image / sequence
- metric: accuracy / F1 / MAE / RMSE / custom
- output format: labels / probabilities / CSV / notebook output
- restrictions: fixed model / no downloads / local weights only

Work in this order:

1. understand metric and data
2. build the cheapest correct baseline
3. validate locally
4. submit early
5. improve only if there is clear signal

2. Problem -> First Baseline

Problem pattern	First baseline
tabular classification	`XGBoost` or `LogisticRegression`
tabular regression	`XGBoost`, `Ridge`, `LinearRegression`
embeddings given	treat embeddings as normal dense features
text classification	`TF-IDF + LogisticRegression`
image classification, small dataset	pretrained `resnet18` with frozen backbone
custom small neural net task	simple MLP + Adam/AdamW + correct loss
unlabeled vectors	`KMeans` + PCA visualization
margin-friendly smaller classification dataset	`SVM` after scaling
assignment / scheduling / constraints	`ortools` / `pulp` / `z3`

3. Data Processing

3.1 Load and inspect fast

```
import pandas as pd
import numpy as np

# Load the training table.
df = pd.read_csv("train.csv")

# Basic shape: rows, columns.
print(df.shape)

# First few rows to inspect column names and obvious issues.
print(df.head())

# Dtypes help you separate numeric vs categorical columns.
print(df.dtypes)

# Missing values report: start with the worst columns.
print(df.isna().sum().sort_values(ascending=False).head(20))
```

3.2 Train/validation split

```
from sklearn.model_selection import train_test_split

# Separate features from target.
X = df.drop(columns=["target"])
y = df["target"]

# Stratify for classification when the target is class-like.
# This keeps class ratios similar in train and validation.
X_train, X_val, y_train, y_val = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y if y.nunique() < 20 else None
)
```

3.3 Missing values

```
# Numeric columns: median is a safe cheap default.
num_cols = X_train.select_dtypes(include=["number"]).columns

# Non-numeric columns: usually categorical or text-like.
cat_cols = X_train.select_dtypes(exclude=["number"]).columns

for c in num_cols:
    med = X_train[c].median()
    X_train[c] = X_train[c].fillna(med)
    X_val[c] = X_val[c].fillna(med)

for c in cat_cols:
    # Use a visible placeholder instead of dropping rows.
    X_train[c] = X_train[c].fillna("missing")
    X_val[c] = X_val[c].fillna("missing")
```

3.4 Encoding and scaling

Rules:

- scale for linear models, SVM, KMeans, neural nets
- do not bother scaling for trees and boosting
- one-hot is fine for linear models and neural nets
- for text use TF-IDF, not raw strings

```
from sklearn.preprocessing import StandardScaler

# Fit only on train data to avoid leakage.
scaler = StandardScaler()

# Scale numeric features only.
Xtr_num = scaler.fit_transform(X_train[num_cols])
Xva_num = scaler.transform(X_val[num_cols])
```

```
# One-hot encode categoricals when using linear models / MLPs.
X_train_oh = pd.get_dummies(X_train, dtype=float)
X_val_oh = pd.get_dummies(X_val, dtype=float)

# Align columns because validation may miss categories seen in train or vice versa.
X_val_oh = X_val_oh.reindex(columns=X_train_oh.columns, fill_value=0)
```

3.5 Metrics

```
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    mean_absolute_error,
    mean_squared_error,
    confusion_matrix,
    classification_report,
)
```

3.6 NumPy quick reference

I checked the available NumPy material in:

- `Numpy Tutorial/numpy_tutorial.ipynb`

I also checked `resourses/`, but that folder currently contains:

- `resourses/notes_chapter_Convolutional_Neural_Networks.pdf`

So for NumPy, the useful local source is the tutorial notebook.

Array creation and dtype

```
import numpy as np

# Create a 1D array from a Python list.
x = np.array([1, 2, 3, 4], dtype=np.float32)

# Range creation: start, stop, step.
a = np.arange(0, 10, 2)

# Evenly spaced points between two values.
b = np.linspace(0.0, 1.0, 5)

# All zeros, ones, constant-filled arrays.
z = np.zeros((3, 4))
o = np.ones((2, 2))
f = np.full((2, 3), 7.0)

# Identity matrix.
I = np.eye(4)

print(x.dtype)
print(a)
print(b)
```

Shape, dimensions, reshape, transpose

```
# 2D matrix with 3 rows and 4 columns.
X = np.arange(12).reshape(3, 4)

# Basic array information.
print(X.shape)   # (3, 4)
print(X.ndim)    # number of dimensions
print(X.size)    # total number of elements

# Flatten to 1D.
flat = X.reshape(-1)

# Transpose rows ⟷ columns.
XT = X.T

# Add a new axis: useful for broadcasting.
col = np.arange(3).reshape(-1, 1)
row = np.arange(4).reshape(1, -1)
```

Indexing, slicing, masking

```
X = np.arange(20).reshape(4, 5)

# Single element.
print(X[2, 3])

# Row slice.
print(X[1:3])

# Column slice.
print(X[:, 2])

# Submatrix.
print(X[1:3, 2:5])

# Boolean mask.
mask = X % 2 == 0
evens = X[mask]

# Find coordinates where condition holds.
coords = np.argwhere(X > 10)

# Conditional replacement.
Y = np.where(X > 10, 1, 0)
```

Concatenate, split, stack

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# End-to-end concatenation.
ab = np.concatenate([a, b])

# Horizontal / vertical stack for matrices.
A = np.ones((2, 3))
B = np.zeros((2, 3))

H = np.hstack([A, B])   # shape (2, 6)
V = np.vstack([A, B])   # shape (4, 3)
C = np.column_stack([a, b])  # columns from 1D arrays

# Split arrays into pieces.
parts = np.array_split(np.arange(10), 3)
```

Arithmetic, broadcasting, clipping

```
x = np.array([1, 2, 3], dtype=np.float32)
y = np.array([10, 20, 30], dtype=np.float32)

# Elementwise arithmetic.
print(x + y)
print(x * y)
print(x / y)

# Scalar broadcasting.
print(x + 5)

# Broadcast row + column to a matrix.
row = np.arange(4).reshape(1, 4)
col = np.arange(3).reshape(3, 1)
grid = row + col

# Clamp values into a safe range.
safe = np.clip(grid, 0, 4)
```

Statistics you will actually use

```
X = np.arange(12).reshape(3, 4).astype(np.float32)

# Global statistics.
print(np.mean(X))
print(np.std(X))
print(np.var(X))
print(np.min(X))
print(np.max(X))
print(np.sum(X))

# Per-axis statistics.
print(np.mean(X, axis=0))   # column means
print(np.mean(X, axis=1))   # row means

# Robust statistics.
print(np.median(X))
print(np.quantile(X, 0.25))
print(np.percentile(X, 90))
```

Sorting and ranking

```
x = np.array([5, 1, 9, 3])

# Sorted values.
print(np.sort(x))

# Index of max / min.
print(np.argmax(x))
print(np.argmin(x))

# Ranking from largest to smallest.
order = np.argsort(-x)
print(order)

# Search insertion position in sorted array.
sorted_x = np.sort(x)
pos = np.searchsorted(sorted_x, 4)
print(pos)
```

NaN / inf safety

```
x = np.array([1.0, np.nan, np.inf, 4.0])

# Detect invalid values before model training.
print(np.isnan(x))
print(np.isinf(x))
print(np.any(np.isnan(x)))
print(np.all(np.isfinite(x)) if hasattr(np, 'isfinite') else 'use isnan/isinf')
```

Linear algebra

```
A = np.array([[1.0, 2.0], [3.0, 4.0]])
B = np.array([[5.0], [6.0]])

# Matrix multiplication.
print(A @ B)
print(np.dot(A, B))

# Useful norms.
print(np.linalg.norm(A))
print(np.linalg.norm(A, axis=1))  # row norms

# Solve / inspect.
print(np.linalg.det(A))
print(np.linalg.inv(A))
```

Trig / exp / log

```
x = np.linspace(0, np.pi, 5)

print(np.sin(x))
print(np.cos(x))
print(np.exp(x))
print(np.log(np.array([1.0, 2.0, 4.0])))
print(np.sqrt(np.array([1.0, 4.0, 9.0])))
```

NumPy functions that appeared in the local tutorial

High-yield ones from `Numpy Tutorial/numpy_tutorial.ipynb`:

- `np.array`
- `np.arange`
- `np.linspace`
- `np.zeros`, `np.ones`, `np.full`, `np.eye`
- `np.reshape`
- `np.concatenate`, `np.hstack`, `np.vstack`, `np.column_stack`, `np.split`, `np.array_split`
- `np.where`, `np.argwhere`
- `np.sort`, `np.argsort`, `np.argmax`, `np.argmin`, `np.searchsorted`
- `np.mean`, `np.median`, `np.std`, `np.var`, `np.min`, `np.max`, `np.sum`
- `np.quantile`, `np.percentile`
- `np.isnan`, `np.isinf`, `np.any`, `np.all`
- `np.dot`
- `np.linalg.norm`, `np.linalg.inv`, `np.linalg.det`
- `np.sin`, `np.cos`, `np.exp`, `np.log`, `np.sqrt`
- `np.clip`, `np.unique`

NumPy contest advice

- if shapes are confusing, print `.shape` after every transformation
- prefer vectorized operations over Python loops
- remember `axis=0` usually means "down the rows / per column"
- remember `axis=1` usually means "across columns / per row"
- use boolean masks instead of slow manual loops
- use `astype(np.float32)` before sending arrays to PyTorch

4. Classical Machine Learning

4.1 Linear Regression / Ridge / Lasso

Use when the target is continuous.

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Ridge is often safer than plain LinearRegression because it regularizes weights.
model = Ridge(alpha=1.0)

# Train on the scaled numeric matrix.
model.fit(Xtr_num, y_train)

# Predict on validation data.
pred = model.predict(Xva_num)

# MAE is robust and easy to interpret.
mae = mean_absolute_error(y_val, pred)

# RMSE punishes large mistakes more strongly.
rmse = mean_squared_error(y_val, pred, squared=False)

print("MAE:", mae)
print("RMSE:", rmse)
```

4.2 Logistic Regression

Use for classification, especially tabular data or TF-IDF features.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score

# class_weight="balanced" is often useful when classes are imbalanced.
model = LogisticRegression(
    max_iter=2000,
    n_jobs=-1,
    class_weight="balanced"
)

# Train on scaled or sparse features.
model.fit(Xtr_num, y_train)

# Class predictions.
pred = model.predict(Xva_num)

print("acc:", accuracy_score(y_val, pred))
print("f1:", f1_score(y_val, pred, average="weighted"))
```

4.3 Decision Tree

Good cheap baseline. Easy to overfit.

```
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

# Keep depth limited at first.
model = DecisionTreeClassifier(
    max_depth=6,
    min_samples_leaf=5,
    random_state=42
)

# Tree models can use the original tabular data directly.
model.fit(X_train, y_train)
pred = model.predict(X_val)
```

Notes:

- increase `max_depth` only if clearly underfitting
- use `min_samples_leaf` to reduce noisy splits
- trees do not need scaling

4.4 XGBoost

Very strong default for tabular problems.

```
from xgboost import XGBClassifier, XGBRegressor

# Strong default classifier for many tabular tasks.
model = XGBClassifier(
    n_estimators=400,        # more trees, smaller steps
    max_depth=6,             # not too deep
    learning_rate=0.05,      # safer than 0.3
    subsample=0.9,           # mild row sampling
    colsample_bytree=0.9,    # mild feature sampling
    eval_metric="mlogloss",  # avoid warning + set classification metric
    random_state=42
)

# X_train_encoded should be numeric only.
model.fit(X_train_encoded, y_train)
pred = model.predict(X_val_encoded)
```

```
# Strong default regressor for tabular regression.
reg = XGBRegressor(
    n_estimators=500,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.9,
    colsample_bytree=0.9,
    random_state=42
)

reg.fit(X_train_encoded, y_train)
pred = reg.predict(X_val_encoded)
```

4.5 KMeans

Use for unsupervised clustering.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

# KMeans is distance-based, so scaling is important.
X_scaled = scaler.fit_transform(X)

best_k = None
best_score = -1

for k in range(2, 11):
    # n_init=20 gives more stable clustering.
    km = KMeans(n_clusters=k, n_init=20, random_state=42)
    labels = km.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)

    if score > best_score:
        best_score = score
        best_k = k

print("best_k:", best_k, "silhouette:", best_score)

# PCA is a fast way to visualize high-dimensional clusters.
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_scaled)
```

4.6 SVM

Good for smaller datasets. Do not start here if the dataset is huge.

```
from sklearn.svm import SVC, SVR

# RBF kernel is a common nonlinear baseline.
clf = SVC(
    C=1.0,
    kernel="rbf",
    gamma="scale",
    class_weight="balanced"
)

# SVM needs scaled features.
clf.fit(Xtr_num, y_train)
pred = clf.predict(Xva_num)
```

4.7 Common pandas / sklearn errors and fast fixes

`ValueError: could not convert string to float`

Meaning:

- a numeric model received raw text or categorical columns

Fast check:

```
print(X_train.select_dtypes(exclude=["number"]).columns.tolist())
```

Fix:

```
# Keep only numeric columns, or encode categoricals explicitly.
X_train_oh = pd.get_dummies(X_train, dtype=float)
X_val_oh = pd.get_dummies(X_val, dtype=float)

# Validation must have the exact same columns as training.
X_val_oh = X_val_oh.reindex(columns=X_train_oh.columns, fill_value=0)
```

`Input X contains NaN`

Meaning:

- the estimator does not accept missing values directly

Fix:

```
# Median for numeric columns is the safest cheap default.
for c in X_train.select_dtypes(include=["number"]).columns:
    med = X_train[c].median()
    X_train[c] = X_train[c].fillna(med)
    X_val[c] = X_val[c].fillna(med)
```

Rule:

- fit missing-value handling on train only
- apply the same fill values to validation and test

`Found input variables with inconsistent numbers of samples`

Meaning:

- `X` and `y` no longer have the same number of rows
- common cause: filtering rows in `X` but not in `y`

Fix:

```
print(len(X), len(y))

# Build one mask, then apply it to both X and y.
mask = X.notna().all(axis=1)
X = X.loc[mask].reset_index(drop=True)
y = y.loc[mask].reset_index(drop=True)
```

`The feature names should match those that were passed during fit`

Meaning:

- train and validation columns differ
- common after one-hot encoding or manual column selection

Fix:

```
# Force the exact training column order before predict().
X_val_oh = X_val_oh.reindex(columns=X_train_oh.columns, fill_value=0)
pred = clf.predict(X_val_oh)
```

`X has n features, but model expects m features`

Meaning:

- you trained on one feature matrix and predicted on another one with a different width

Fast check:

```
print(X_train_oh.shape)
print(X_val_oh.shape)
```

Most common fixes:

- reindex validation columns to training columns
- do not fit PCA / scaler / vectorizer separately on validation
- do not drop columns on only one split

`Unknown label type: continuous`

Meaning:

- you used a classifier for a regression target

Fix:

- if the target is a real number, use regressor classes like `Ridge`, `RandomForestRegressor`, `XGBRegressor`
- if it is classification, make sure labels are true class ids, not floats like `0.0, 1.0, 2.0`

`This solver needs samples of at least 2 classes`

Meaning:

- one split contains only one class
- common on tiny datasets or non-stratified splits

Fix:

```
print(y_train.value_counts())

X_train, X_val, y_train, y_val = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

5. PyTorch Simple Models

Use the exact training function style you already use.

5.1 Tabular MLP for classification with `nn.Module`

```
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

# Pick GPU if available.
device = "cuda" if torch.cuda.is_available() else "cpu"

# Convert NumPy arrays to PyTorch tensors.
Xtr = torch.tensor(X_train_np, dtype=torch.float32)
Xva = torch.tensor(X_val_np, dtype=torch.float32)
ytr = torch.tensor(y_train_np, dtype=torch.long)   # long for CrossEntropyLoss
yva = torch.tensor(y_val_np, dtype=torch.long)

# Build datasets and loaders.
train_loader = DataLoader(TensorDataset(Xtr, ytr), batch_size=128, shuffle=True)
val_loader = DataLoader(TensorDataset(Xva, yva), batch_size=256, shuffle=False)

class TabularClassifier(nn.Module):
    def __init__(self, input_dim, num_classes):
        super(TabularClassifier, self).__init__()
        # First hidden layer: expand feature space.
        self.fc1 = nn.Linear(input_dim, 256)
        # Second hidden layer: compress toward useful representation.
        self.fc2 = nn.Linear(256, 128)
        # Final layer outputs raw logits, one per class.
        self.fc3 = nn.Linear(128, num_classes)
        # ReLU is the standard safe activation.
        self.relu = nn.ReLU()
        # Dropout helps fight overfitting.
        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        # Hidden block 1.
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)

        # Hidden block 2.
        x = self.fc2(x)
        x = self.relu(x)
        x = self.dropout(x)

        # Final logits. No softmax here because CrossEntropyLoss already handles it.
        x = self.fc3(x)
        return x

model = TabularClassifier(input_dim=Xtr.shape[1], num_classes=num_classes).to(device)

# Standard classification loss.
criterion = nn.CrossEntropyLoss()

# AdamW is a strong default optimizer.
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
```

5.2 Tabular MLP for regression with `nn.Module`

```
class TabularRegressor(nn.Module):
    def __init__(self, input_dim):
        super(TabularRegressor, self).__init__()
        # Same structure as classification, but final output is a single number.
        self.fc1 = nn.Linear(input_dim, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 1)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)

        x = self.fc2(x)
        x = self.relu(x)

        # One scalar per sample.
        x = self.fc3(x)
        return x

model = TabularRegressor(input_dim=Xtr.shape[1]).to(device)

# Regression target should be float, not long.
ytr = torch.tensor(y_train_np, dtype=torch.float32)
yva = torch.tensor(y_val_np, dtype=torch.float32)

train_loader = DataLoader(TensorDataset(Xtr, ytr), batch_size=128, shuffle=True)
val_loader = DataLoader(TensorDataset(Xva, yva), batch_size=256, shuffle=False)

# MSE is the default regression loss.
criterion = nn.MSELoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
```

5.3 Training function from `comp/train_func.py`

```
import torch
import copy
import torch.nn.functional as F
import matplotlib.pyplot as plt

def train(model, train_loader, test_loader, optimizer, epochs, criterion):
    """
    Trains a PyTorch model, evaluates it on a test set, and plots the results.
    """
    # Select GPU if available.
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Move model once to the device.
    model = model.to(device, non_blocking=True)

    # Decay LR by a factor of 0.1 every 7 epochs.
    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)

    # Store curves for plotting later.
    train_losses = []
    test_losses = []
    test_accuracies = []

    # Track best validation accuracy and weights.
    best_accuracy = 0
    best_model_wts = copy.deepcopy(model.state_dict())

    # Main training loop.
    for epoch in range(epochs):
        # Put the model in training mode.
        model.train()
        total_loss = 0

        # Training pass over all mini-batches.
        for batch_x, batch_y in train_loader:
            # Move batch to GPU/CPU.
            batch_x = batch_x.to(device, non_blocking=True)
            batch_y = batch_y.to(device, non_blocking=True)

            # Clear old gradients.
            optimizer.zero_grad()

            # Forward pass.
            outputs = model(batch_x)

            # Compute loss.
            loss = criterion(outputs, batch_y)

            # Backward pass.
            loss.backward()

            # Update weights.
            optimizer.step()

            # Accumulate batch loss.
```

```
            total_loss += loss.item()

        # Step the scheduler once per epoch.
        scheduler.step()

        # Average train loss for this epoch.
        train_losses.append(total_loss / len(train_loader))

        # Validation phase.
        model.eval()
        total_test_loss = 0
        correct = 0
        total = 0

        # Turn off gradients during validation.
        with torch.no_grad():
            for batch_x, batch_y in test_loader:
                # Move validation batch to device.
                batch_x = batch_x.to(device)
                batch_y = batch_y.to(device)

                # Forward pass.
                outputs = model(batch_x)

                # Validation loss.
                loss = criterion(outputs, batch_y)
                total_test_loss += loss.item()

                # Predicted class = largest logit.
                _, predicted = torch.max(outputs.data, 1)
                total += batch_y.size(0)
                correct += (predicted == batch_y).sum().item()

        # Average validation loss and accuracy.
        epoch_loss = total_test_loss / len(test_loader)
        epoch_accuracy = correct / total
        test_losses.append(epoch_loss)
        test_accuracies.append(epoch_accuracy)

        # Save best model.
        if epoch_accuracy > best_accuracy:
            best_accuracy = epoch_accuracy
            best_model_wts = copy.deepcopy(model.state_dict())
            best_epoch = epoch + 1

        print(
            f"Epoch {epoch+1}/{epochs}, "
            f"Training Loss: {train_losses[-1]}, "
            f"Testing Loss: {test_losses[-1]}, "
            f"Testing Accuracy: {epoch_accuracy:.4f}"
        )

    # Restore best validation checkpoint.
    model.load_state_dict(best_model_wts)
    print(f"Loaded the best model from epoch {best_epoch} with Testing Accuracy:
{best_accuracy:.4f}")
```

```
    # Plot loss and accuracy curves.
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(range(1, epochs+1), train_losses, label='Training Loss')
    plt.plot(range(1, epochs+1), test_losses, label='Testing Loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(range(1, epochs+1), test_accuracies, label='Testing Accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()

    plt.show()
```

5.4 How to call that training function

```
# Build model, loss, optimizer first.
model = TabularClassifier(input_dim=Xtr.shape[1], num_classes=num_classes).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)

# Then train.
train(
    model=model,
    train_loader=train_loader,
    test_loader=val_loader,
    optimizer=optimizer,
    epochs=20,
    criterion=criterion
)
```

6. PyTorch Optimization Notes

6.1 Correct output / target / loss combinations

Task	Output	Target dtype	Loss
regression	1 float per sample	`float32`	`MSELoss`, `L1Loss`, `SmoothL1Loss`
multiclass classification	`num_classes` logits	`long`	`CrossEntropyLoss`
binary classification	1 logit	`float32` in `{0,1}`	`BCEWithLogitsLoss`

6.2 High-value rules

- use `AdamW` first
- try `lr=1e-3` for small models
- try `lr=1e-4` when finetuning pretrained backbones
- if loss explodes, lower LR
- if model does not learn, overfit 8 samples

- save the best checkpoint, not just the last epoch

6.3 Tiny-batch overfit test

If this cannot drive loss very low, your model, labels, or loss setup is wrong.

```
# Pick 8 training examples only.
tiny_x = Xtr[:8].to(device)
tiny_y = ytr[:8].to(device)

for step in range(200):
    # Forward pass.
    pred = model(tiny_x)

    # If regression model returns shape (N, 1), remove the last dim.
    if pred.ndim == 2 and pred.shape[1] == 1:
        pred = pred.squeeze(-1)

    # Compute loss on the tiny batch.
    loss = criterion(pred, tiny_y)

    # Standard optimization step order.
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

print("tiny-batch loss:", loss.item())
```

6.4 DataLoader performance

From your local notes:

- `pin_memory=True` helps when you also use `non_blocking=True`
- too many workers can waste RAM
- `4` to `8` workers is often enough

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_ds,
    batch_size=128,          # medium batch size
    shuffle=True,            # shuffle training data
    num_workers=6,           # not too small, not too large
    pin_memory=True,         # helps faster host→GPU copies
    persistent_workers=True  # avoid worker restart every epoch
)

for xb, yb in train_loader:
    # non_blocking works best with pin_memory=True.
    xb = xb.to(device, non_blocking=True)
    yb = yb.to(device, non_blocking=True)
```

6.5 Common PyTorch code errors and how to fix them

`Expected all tensors to be on the same device`

Meaning:

- your model is on GPU but a batch is on CPU, or the opposite

Fix:

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)

for xb, yb in train_loader:
    xb = xb.to(device, non_blocking=True)
    yb = yb.to(device, non_blocking=True)
```

Also remember:

- if you create tensors manually inside training, put them on `device`
- a common mistake is `torch.tensor([target_class])` without `.to(device)`

```
target = torch.tensor([target_class], device=device)
```

`Expected scalar type Long but found Float`

Meaning:

- you are using `CrossEntropyLoss`, but labels are floats

Fix:

```
ytr = torch.tensor(y_train_np, dtype=torch.long)
criterion = nn.CrossEntropyLoss()
```

Rule:

- `CrossEntropyLoss` -> labels must be integer class ids (`long`)
- `MSELoss` / `L1Loss` -> targets must usually be `float32`

`Expected input batch_size to match target batch_size`

Meaning:

- output shape and target shape do not line up

Fix:

```
print("pred shape:", pred.shape)
print("target shape:", yb.shape)
```

Typical expected shapes:

- classification:
- `pred` : `(batch_size, num_classes)`
- `yb` : `(batch_size,)`
- regression:
- `pred` : `(batch_size, 1)` or `(batch_size,)`
- `yb` : `(batch_size,)`

```
pred = model(xb)
pred = pred.squeeze(-1)   # only for regression if shape is (N, 1)
loss = criterion(pred, yb)
```

`mat1 and mat2 shapes cannot be multiplied`

Meaning:

- your `Linear` layer input size is wrong

Fix:

```
print(x.shape)
```

Most common conv-net fix:

```
x = x.view(x.size(0), -1)
```

Best habit:

- print tensor shapes inside `forward()` while debugging
- verify that `nn.Linear(in_features= … )` matches the flattened tensor

`RuntimeError: stack expects each tensor to be equal size`

Example:

```
RuntimeError: stack expects each tensor to be equal size,
but got [10000, 3] at entry 0 and [10000, 2] at entry 1
```

Meaning:

- you used `torch.stack( … )` on tensors with different shapes
- `stack` adds a new dimension, so every input tensor must already have the exact same shape

Why this happens:

- two feature blocks have different numbers of columns
- predictions from different models have different class counts
- one tensor is missing a column / channel / time step
- variable-length sequences were not padded first

Wrong:

```
a = torch.randn(10000, 3)
b = torch.randn(10000, 2)
x = torch.stack([a, b])   # crash
```

Fix 1: if you wanted to combine features side-by-side, use `torch.cat( … , dim=1)`

```
a = torch.randn(10000, 3)
b = torch.randn(10000, 2)

# Result shape: (10000, 5)
x = torch.cat([a, b], dim=1)
```

Fix 2: if you really need `stack`, first make shapes equal

```
a = torch.randn(10000, 3)
b = torch.randn(10000, 3)

# Result shape: (2, 10000, 3)
x = torch.stack([a, b], dim=0)
```

Fix 3: print shapes before stacking

```
print(a.shape)
print(b.shape)
```

Rule:

- `torch.stack([a, b])` requires `a.shape == b.shape`
- `torch.cat([a, b], dim=k)` requires same shape on all dimensions except `dim=k`

Common contest cases:

- combining embeddings from different sources
- combining logits from models with different class counts
- collecting variable-length sequences without padding
- stacking image tensors that were resized differently

Square-painting generalization:

- this also happens when one tensor is the **input** and the other is the **target**
- for example, coordinates may have shape `(N, 2)` while RGB targets have shape `(N, 3)`
- those are different roles, not two copies of the same kind of tensor

Wrong pattern:

```
# inputs: x,y coordinates
inputs.shape == (N, 2)

# targets: r,g,b values
targets.shape == (N, 3)

bad = torch.stack([inputs, targets])   # wrong
```

Why it is wrong:

- `stack` means "same tensor shape, add a new axis"
- but `(N, 2)` and `(N, 3)` are not the same shape
- semantically they should stay separate as `(x, y)` in supervised learning

Correct supervised-learning pattern:

```
from torch.utils.data import TensorDataset, DataLoader

# Keep inputs and targets separate.
dataset = TensorDataset(inputs, targets)
loader = DataLoader(dataset, batch_size=64, shuffle=True)

for batch_x, batch_y in loader:
    pred = model(batch_x)
    loss = criterion(pred, batch_y)
```

If you really mean to build one combined feature matrix, use `cat`, not `stack`:

```
# Only if combining into one feature table actually makes sense.
xy_rgb = torch.cat([inputs, targets], dim=1)   # shape (N, 5)
```

Competition rule:

- different roles -> keep separate
- same role, different feature blocks -> usually `cat`
- new axis over equal-shaped tensors -> `stack`

`CUDA out of memory`

Meaning:

- your batch, model, or image size is too large for the GPU

Fixes:

- reduce `batch_size`
- reduce input resolution
- use a smaller model
- avoid storing unnecessary tensors
- use `torch.no_grad()` in validation

```
batch_size = 32   # try 128 → 64 → 32 → 16
```

Loss becomes `nan`

Meaning:

- unstable optimization, bad preprocessing, or invalid values

Fixes:

- lower learning rate
- check for NaNs in inputs and labels
- normalize inputs
- clip gradients

```
print(torch.isnan(xb).any())
print(torch.isnan(yb).any())

torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

Accuracy stays near random chance

Meaning:

- the task/loss/labels are mismatched, or the model is not learning

Fixes:

- overfit 8 samples
- verify class balance
- verify optimizer order
- verify you are not applying `softmax` before `CrossEntropyLoss`

Wrong:

```
probs = torch.softmax(model(xb), dim=1)
loss = nn.CrossEntropyLoss()(probs, yb)
```

Correct:

```
logits = model(xb)
loss = nn.CrossEntropyLoss()(logits, yb)
```

Regression problem but classification code was reused

Meaning:

- you copied a classification loop into a regression task

Fix:

```
criterion = nn.MSELoss()
ytr = torch.tensor(y_train_np, dtype=torch.float32)
pred = model(xb).squeeze(-1)
loss = criterion(pred, yb)
```

Rule:

- regression = float targets + regression loss
- classification = integer targets + classification loss

`Trying to backward through the graph a second time`

Meaning:

- you reused a tensor with graph history from a previous step

Fix:

```
x = x.detach()
```

Common cases:

- adversarial attacks
- iterative input optimization
- reusing model outputs later without detaching

In-place operation error

Meaning:

- you modified a tensor that autograd still needed

Fix:

```
x_tmp = x.clone().detach()
# modify x_tmp instead of x directly
```

Common cases:

- manual image attacks
- direct optimization on tensors
- custom training logic with partial updates

Validation behaves strangely because Dropout / BatchNorm are wrong

Meaning:

- you forgot to switch between training and evaluation modes

Fix:

```
model.train()   # training
model.eval()    # validation / inference
```

Always combine evaluation with:

```
with torch.no_grad():
    ...
```

Best quick PyTorch debug print block

```
print("xb:", xb.shape, xb.dtype, xb.device)
print("yb:", yb.shape, yb.dtype, yb.device)
pred = model(xb)
print("pred:", pred.shape, pred.dtype, pred.device)
print("pred sample:", pred[:2])
print("target sample:", yb[:10])
```

6.6 More PyTorch errors you may hit

`Target 7 is out of bounds`

Meaning:

- your labels contain a class id outside `[0, num_classes - 1]`
- common when labels start at `1` instead of `0`

Fix:

```
print(ytr.min().item(), ytr.max().item())

# Example: labels were 1..K, convert them to 0..K-1.
ytr = ytr - 1
yva = yva - 1

# Final layer must output one logit per class.
model.fc = nn.Linear(model.fc.in_features, num_classes)
```

`Using a target size ... different to the input size ...` with `BCEWithLogitsLoss`

Meaning:

- binary-classification logits and targets do not have matching shapes

Correct pattern:

```
# One logit per sample.
logits = model(xb).squeeze(1)

# BCE targets must be float 0/1, not long class ids.
yb = yb.float()

loss = nn.BCEWithLogitsLoss()(logits, yb)
```

Rule:

- binary classification with one output neuron -> `BCEWithLogitsLoss`
- multiclass classification with `C` outputs -> `CrossEntropyLoss`

`view size is not compatible with input tensor's size and stride`

Meaning:

- you tried `.view(...)` on a non-contiguous tensor
- common after `transpose`, `permute`, or slicing

Fix:

```
# reshape() is safer when tensor memory is not contiguous.
x = x.reshape(x.size(0), -1)
```

Alternative:

```
x = x.contiguous().view(x.size(0), -1)
```

`Can't call numpy() on Tensor that requires grad`

Meaning:

- you tried to convert a tensor to NumPy before detaching it from autograd

Fix:

```
arr = pred.detach().cpu().numpy()
```

Rule:

- plotting / saving / scikit-learn use -> `tensor.detach().cpu().numpy()`

`DataLoader worker (pid ...) exited unexpectedly`

Meaning:

- worker subprocesses crashed
- on Windows this often comes from notebook multiprocessing issues

Fast fix:

```
# First make it work. Then increase workers again if needed.
train_loader = DataLoader(train_ds, batch_size=128, shuffle=True, num_workers=0)
```

Then check:

- dataset `__getitem__` does not crash
- images/files actually exist
- batch creation does not use unsupported objects
- only raise `num_workers` after the single-process loader works

`element 0 of tensors does not require grad and does not have a grad_fn`

Meaning:

- you broke the graph before `backward()`

Common causes:

- calling `.detach()` too early
- converting to NumPy in the middle of training
- wrapping a tensor with `torch.tensor(existing_tensor)` instead of using it directly

Wrong:

```
pred = model(xb).detach()
loss = criterion(pred, yb)
loss.backward()   # crash
```

Correct:

```
pred = model(xb)
loss = criterion(pred, yb)
loss.backward()
```

7. PyTorch Computer Vision

7.1 Preprocessing

```
from torchvision import transforms as T

train_tf = T.Compose([
    # Random crop introduces small geometric variety.
    T.RandomResizedCrop(224),
    # Horizontal flip is useful for many natural image tasks.
    T.RandomHorizontalFlip(),
    # Mild color jitter helps robustness.
    T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.05),
    # Convert PIL image to tensor in [0, 1].
    T.ToTensor(),
    # ImageNet normalization for pretrained backbones.
    T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

val_tf = T.Compose([
    # Deterministic validation pipeline.
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])
```

7.2 Transfer learning baseline

Best default for small image datasets.

```
import torch
import torch.nn as nn
from torchvision import models

# Load pretrained ResNet18 weights.
model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

# Freeze all backbone weights first.
for p in model.parameters():
    p.requires_grad = False

# Replace the classifier head with one that matches your classes.
model.fc = nn.Linear(model.fc.in_features, num_classes)
model = model.to(device)

# Train only the new head first.
optimizer = torch.optim.AdamW(model.fc.parameters(), lr=1e-3, weight_decay=1e-4)
criterion = nn.CrossEntropyLoss()
```

Later improvements if time allows:

- unfreeze last residual block
- lower LR to `1e-4`
- continue training for a few epochs

7.3 Simple CNN with `nn.Module`

Use when pretrained ImageNet features do not fit the domain or when the task is intentionally small/simple.

```
import torch
import torch.nn as nn

class SmallCNN(nn.Module):
    def __init__(self, n_cls):
        super(SmallCNN, self).__init__()

        # First conv block: learn low-level edges and textures.
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 32, kernel_size=3, padding=1)

        # Second conv block: learn more abstract patterns.
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv4 = nn.Conv2d(64, 64, kernel_size=3, padding=1)

        # Pooling reduces spatial size and computation.
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        # Adaptive pooling makes the model robust to final feature-map size.
        self.gap = nn.AdaptiveAvgPool2d(1)

        # Fully connected classifier head.
        self.fc1 = nn.Linear(64, 128)
        self.fc2 = nn.Linear(128, n_cls)

        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.3)

    def forward(self, x):
        # Block 1.
        x = self.conv1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.relu(x)
        x = self.pool(x)

        # Block 2.
        x = self.conv3(x)
        x = self.relu(x)
        x = self.conv4(x)
        x = self.relu(x)
        x = self.pool(x)

        # Global average pooling → shape becomes (N, 64, 1, 1).
        x = self.gap(x)

        # Flatten to (N, 64).
        x = x.view(x.size(0), -1)

        # Classifier head.
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
```

```
        # Raw logits out.
        return x
```

7.4 Vision reminders

- PIL / NumPy image shape: `(H, W, C)`
- PyTorch image shape: `(C, H, W)`
- PyTorch batch shape: `(N, C, H, W)`
- OpenCV loads color images in **BGR**, not RGB
- for tiny image datasets, augmentation often matters more than a bigger model

7.5 Vision-specific failure patterns

`expected input to have 3 channels, but got 1 channels instead`

Meaning:

- the backbone expects RGB input but your data is grayscale

Fix options:

- convert grayscale images to 3 channels during preprocessing
- or change the first conv layer if the architecture is allowed

```
# Repeat one grayscale channel into fake RGB.
x = x.repeat(1, 3, 1, 1)
```

Images look correct to you, but training is terrible

Common causes:

- you forgot ImageNet normalization for pretrained backbones
- train and validation transforms are inconsistent
- labels from folder names were mapped in the wrong order

Quick checks:

```
print(train_ds.class_to_idx)
print(xb.shape, xb.min().item(), xb.max().item())
```

`mat1 and mat2 shapes cannot be multiplied` after a conv stack

Meaning:

- the flattened tensor does not match the first linear layer

Fix:

```
# Print the tensor shape right before flattening.
print(x.shape)
x = x.reshape(x.size(0), -1)
```

8. Embeddings and Text

8.1 Embeddings already given

Treat embeddings like normal dense numeric features.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load embedding matrix and labels.
X = np.load("train_embeddings.npy")
y = np.load("train_labels.npy")

# Train/validation split first.
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Logistic regression is a very strong cheap baseline on embeddings.
clf = LogisticRegression(max_iter=2000, n_jobs=-1)
clf.fit(X_train, y_train)

pred = clf.predict(X_val)
print("acc:", accuracy_score(y_val, pred))
```

```
from xgboost import XGBRegressor

# For embedding regression tasks, boosting is usually strong.
reg = XGBRegressor(
    n_estimators=500,
    max_depth=6,
    learning_rate=0.05,
    random_state=42
)

reg.fit(X_train, y_train)
pred = reg.predict(X_val)
```

8.2 Cosine similarity retrieval

```
import numpy as np

def l2norm(x):
    # Normalize rows to unit length so cosine similarity becomes dot product.
    return x / (np.linalg.norm(x, axis=1, keepdims=True) + 1e-9)

# Normalize corpus/document embeddings.
doc_emb = l2norm(doc_emb)

# Normalize query embedding too.
q_emb = l2norm(q_emb.reshape(1, -1))[0]

# Cosine similarities because all vectors are unit-normalized.
sims = doc_emb @ q_emb

# Top indices with highest similarity.
topk = np.argsort(-sims)[:10]
```

8.3 TF-IDF + LogisticRegression

This should usually be your first text baseline.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from scipy.sparse import hstack

# Word n-grams capture normal word patterns.
vec_word = TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=3,
    max_df=0.95,
    sublinear_tf=True,
    max_features=200_000,
)

# Character n-grams are very useful for noisy text, spelling variation, and Greeklish.
vec_char = TfidfVectorizer(
    analyzer="char_wb",
    ngram_range=(3, 5),
    min_df=3,
)

# Fit on training text only.
Xw_tr = vec_word.fit_transform(train_texts)
Xw_va = vec_word.transform(val_texts)

Xc_tr = vec_char.fit_transform(train_texts)
Xc_va = vec_char.transform(val_texts)

# Combine word and character features.
Xtr = hstack([Xw_tr, Xc_tr]).tocsr()
Xva = hstack([Xw_va, Xc_va]).tocsr()

# Logistic regression is a strong sparse-text baseline.
clf = LogisticRegression(
    max_iter=2000,
    class_weight="balanced",
    n_jobs=-1
)

clf.fit(Xtr, y_train)
pred = clf.predict(Xva)
```

8.4 `nn.Embedding`

```
import torch
import torch.nn as nn

# vocab_size = number of token ids
# embedding_dim = vector size per token
embed = nn.Embedding(num_embeddings=vocab_size, embedding_dim=128)

# Each integer is a token id.
token_ids = torch.tensor([
    [1, 4, 7],
    [2, 9, 0]
])

# Output shape: (batch, seq_len, embedding_dim)
out = embed(token_ids)
print(out.shape)
```

8.5 Offline warning

- do not assume you can download Hugging Face models
- use pretrained models only if weights are already available locally
- if not sure, use TF-IDF or provided embeddings first

8.6 Text / embedding failure patterns

Text model validation is suspiciously too good

Common cause:

- `TfidfVectorizer` or tokenizer was fit on the full dataset before the split

Rule:

- split first
- fit vectorizer on train only
- transform validation/test with the fitted train vectorizer

Sparse text pipeline suddenly uses huge RAM

Common cause:

- converting sparse TF-IDF matrices to dense arrays

Avoid:

```
X_dense = X_tfidf.toarray()   # often unnecessary and expensive
```

Prefer:

- models that accept sparse input directly, such as `LogisticRegression`, `LinearSVC`, `SGDClassifier`

Retrieval results look wrong although shapes are fine

Common cause:

- cosine similarity was used without normalizing embeddings

Fix:

```
E = E / np.linalg.norm(E, axis=1, keepdims=True)
q = q / np.linalg.norm(q)
scores = E @ q
```

`IndexError: index out of range in self` with `nn.Embedding`

Meaning:

- some token id is `≥ vocab_size`

Fix:

```
print(tokens.min().item(), tokens.max().item())
embedding = nn.Embedding(vocab_size, embedding_dim)
```

9. Optimization / Solver Problems

If the problem is discrete and constraint-heavy, think solver before ML.

9.1 PuLP assignment example

```
import pulp

N, M = 5, 3
cost = [
    [4, 2, 8],
    [1, 9, 3],
    [5, 4, 6],
    [7, 2, 3],
    [2, 8, 1]
]

# Minimize total assignment cost.
prob = pulp.LpProblem("assignment", pulp.LpMinimize)

# Binary variable x[i][j] = 1 if item i goes to slot j.
x = [[pulp.LpVariable(f"x_{i}_{j}", cat="Binary") for j in range(M)] for i in range(N)]

# Objective.
prob += pulp.lpSum(cost[i][j] * x[i][j] for i in range(N) for j in range(M))

# Each item must be assigned exactly once.
for i in range(N):
    prob += pulp.lpSum(x[i][j] for j in range(M)) == 1

# Each slot has capacity ≤ 2.
for j in range(M):
    prob += pulp.lpSum(x[i][j] for i in range(N)) ≤ 2

# Solve silently.
prob.solve(pulp.PULP_CBC_CMD(msg=False))
```

9.2 OR-Tools CP-SAT skeleton

```
from ortools.sat.python import cp_model

# Create a CP-SAT model.
model = cp_model.CpModel()

# Add variables and constraints here.

# Create solver.
solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 10

# Solve.
status = solver.Solve(model)
```

10. Debugging Checklist

10.1 Find the broken stage first

Do not debug everything at once. First identify the failing stage:

- load: files missing, wrong paths, wrong CSV separator, image decode fails
- split: train/validation misaligned, class missing in one split, leakage
- features: NaNs, raw strings, unseen categories, wrong normalization
- model input: wrong tensor shape, wrong dtype, wrong device
- training: loss does not fall, gradients break, LR too high, OOM
- validation: `model.eval()` missing, wrong metric, shuffled labels
- export/submission: wrong column names, wrong file name, wrong JSON/CSV format

10.2 60-second debug routine

When the model is broken:

1. print shapes and dtypes
2. print 3 raw samples and 3 targets
3. verify the metric and required submission format
4. verify train/validation split and target balance
5. run the cheapest baseline
6. overfit a tiny batch
7. move LR by `x10` and `/10`
8. inspect one prediction manually

10.3 Useful prints

```
# Table / NumPy sanity.
print(X.shape, y.shape)
print(type(X), type(y))
print(X.dtype, y.dtype)

# Raw examples reveal parsing bugs fast.
print(df.head())

# Target balance matters for both splits and metric choice.
print(np.unique(y, return_counts=True))
```

```
# PyTorch sanity block.
print("xb:", xb.shape, xb.dtype, xb.device)
print("yb:", yb.shape, yb.dtype, yb.device)
pred = model(xb)
print("pred:", pred.shape, pred.dtype, pred.device)
print("pred sample:", pred[:2])
print("target sample:", yb[:10])
```

10.4 Common failures by segment

Data loading

- file exists but you are in the wrong working directory
- CSV delimiter is not `,`
- image paths are relative to a different folder
- labels were read as strings with spaces

Quick checks:

```
print(df.shape)
print(df.columns.tolist())
print(df.dtypes)
```

Split / leakage

- scaling, PCA, TF-IDF, or imputation fit on full data instead of train only
- duplicates from train leaking into validation
- target accidentally left inside features

Quick checks:

```
print("target in X:", "target" in X.columns)
print("train rows:", len(X_train), "val rows:", len(X_val))
```

Modeling

- classifier used for regression
- regression loss used with integer class labels
- `softmax` added before `CrossEntropyLoss`
- binary task coded as multiclass or the reverse

Optimization

- LR too high -> loss becomes `nan` or explodes
- LR too low -> loss barely changes
- batch too large -> OOM
- no tiny-batch overfit -> setup bug likely remains

Submission / export

- row order changed
- id column missing
- probabilities required, but class ids submitted
- one file or level missing from exported JSON

Quick checks:

```
print(submission.head())
print(submission.shape)
print(submission.columns.tolist())
```

10.5 Mandatory optimizer order

```
# Clear old gradients first.
optimizer.zero_grad()

# Compute gradients.
loss.backward()

# Apply one optimization step.
optimizer.step()
```

10.6 Submission / export sanity block

```
# Use right before final write.
print("rows:", len(submission))
print("columns:", submission.columns.tolist())
print(submission.head(3))

# For JSON answers, also inspect one entry manually.
print(list(result.keys())[:3])
```

11. Worked Problems From This Repo

These are not just references. They are solved patterns you can reuse.

11.1 `emotions.ipynb`: sparse adversarial attack on emotion classifier

Problem pattern:

- input is a grayscale `112 x 112` face image
- model is fixed
- goal is to change the predicted emotion with very small `L1` pixel edits

What the notebook does:

- loads a pretrained grayscale-adapted `ShuffleNet`

- wraps preprocessing inside the model
- uses gradients with respect to the image
- changes only the highest-impact pixels
- moves each chosen pixel by only `+1` or `-1`
- clamps pixels into `[0, 255]`

Key idea:

- do **not** optimize all pixels with large continuous changes
- instead, rank pixels by `abs(gradient)` and edit only a few each step

Important pattern:

```
# Gradient-based sparse pixel attack:
# 1. compute loss toward target class
# 2. backprop into the image
# 3. sort pixels by gradient magnitude
# 4. change only the top few pixels by +/-1

output = model(adv.clamp(0, 255))
loss = F.cross_entropy(output, torch.tensor([target_class]))
model.zero_grad()
loss.backward()

grad = adv.grad.detach()
indices = torch.argsort(grad.view(-1).abs(), descending=True)
```

What to remember:

- `targeted` attack: push image toward a chosen target class
- `untargeted` attack: push image away from the original class
- use `.clone().detach()` often to avoid autograd mistakes
- compare images after `.round()` because submission is integer-valued

11.2 `challenges/nextmovie.ipynb`: ranking by optimized query embedding

Problem pattern:

- you are given movie embeddings
- you must produce one query vector so that movies rank in a required order

Cheap solution:

- build the query as a weighted sum of the target embeddings
- subtract the mean embedding of non-target movies
- normalize the final query

```
# Target movie embeddings.
T = E[targets]

# Non-target embeddings.
others = E[[i for i in range(len(E)) if i not in targets]]

# Larger weights for earlier desired ranks.
weights = torch.tensor([32., 16., 8., 4., 2.])

# Pull target movies closer.
emb = (weights[:, None] * T).sum(dim=0)

# Push the rest away.
emb = emb - 4 * others.mean(dim=0)

# Normalize for cosine similarity.
emb = emb / emb.norm()
```

Better solution from the notebook:

- optimize the query vector directly with `Adam`
- define loss terms that force:
- `target1 > target2 > ... > target5`
- all targets > all non-targets
- use cosine similarity and exponential hinge-like penalties

What to remember:

- ranking tasks on embeddings often reduce to **query vector design**
- normalization is critical
- weighted sums are a strong baseline
- direct optimization over the query is often enough; you do not need to retrain a model

11.3 `challenges/captcha.ipynb`: solve arithmetic CAPTCHA by segmenting and classifying digits

Problem pattern:

- one image contains `digit digit operator digit digit`
- output is the computed arithmetic result

The notebook solution:

- train MNIST digit classifiers first
- train one classifier on clean MNIST
- train another classifier on noisy/augmented MNIST
- split the CAPTCHA image into 5 fixed `28x28` slices
- classify each digit slice independently
- detect operator with a cheap heuristic instead of training another model

Digit model used:

```
class ConvNet(nn.Module):
    def __init__(self):
        super(ConvNet, self).__init__()
        self.conv1 = nn.Conv2d(1, 10, kernel_size=5)
        self.conv2 = nn.Conv2d(10, 20, kernel_size=5)
        self.fc1 = nn.Linear(320, 50)
        self.fc2 = nn.Linear(50, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.conv1(x)
        x = self.relu(x)
        x = F.max_pool2d(x, 2)
        x = self.conv2(x)
        x = self.relu(x)
        x = F.max_pool2d(x, 2)
        x = x.view(-1, 320)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

Image splitting pattern:

```
# Full CAPTCHA has shape roughly (1, 28, 140).
# Split across width into 5 pieces of width 28.
digit1 = eq[:, :, :, 0:28]
digit2 = eq[:, :, :, 28:56]
op     = eq[:, :, :, 56:84]
digit3 = eq[:, :, :, 84:112]
digit4 = eq[:, :, :, 112:140]
```

Operator trick:

- `+` has a vertical stroke
- `-` mostly does not
- the notebook uses pixel sums in a central vertical strip to distinguish them

Noise trick:

- for noisy CAPTCHAs, it applies largest-connected-component extraction
- this removes isolated noise while keeping the main digit structure

What to remember:

- if positions are fixed, segmentation can be a simple slice, not a detector
- if only one symbol differs structurally, a heuristic may beat training another model
- train on noisy augmentations if the test data is noisy

11.4 `pdtn2025/deepfakes_final.ipynb`: binary image classification with a required backbone

Problem pattern:

- image classification
- architecture may be constrained
- dataset is split into train / validation folders

What the notebook does:

- uses `ImageFolder`-style dataset setup
- normalizes images with ImageNet statistics
- uses `torchvision.models.shufflenet_v2_x1_0`
- swaps the last classifier layer to `2` outputs
- optionally loads ImageNet pretrained weights
- trains with `Adam`, low learning rate, and validation tracking

Important pattern:

```
# Load the required backbone. Pretrained weights are the strongest cheap start.
model = models.shufflenet_v2_x1_0(
    weights=models.ShuffleNet_V2_X1_0_Weights.IMAGENET1K_V1
)

# Replace only the final classifier so logits match the two contest classes.
model.fc = torch.nn.Linear(model.fc.in_features, 2)

# Small LR because the pretrained backbone already knows useful visual features.
optimizer = optim.Adam(model.parameters(), lr=1e-4, weight_decay=1e-4)
```

What to remember:

- when the architecture is fixed, the main gains come from:
- pretrained initialization
- correct transforms
- optimizer / LR choice
- enough validation monitoring

11.5 `pdtn2025/Αντίγραφο_embeddings_greedy_torch (1).ipynb`: word-chain search on embeddings

Problem pattern:

- words are nodes
- cosine similarity defines closeness
- you need a chain from one word to another

What the notebook does:

- reads embeddings
- L2-normalizes them
- computes pairwise similarities with matrix multiplication
- converts similarity to a discrete distance score
- tests greedy strategies

Important pattern:

```
# Similarity to all words at once.
sim_full = torch.matmul(embeddings_tensor, torch.t(embeddings_tensor))

# Greedy next-step candidates.
greedy_best = torch.topk(sim_full, k=32057, axis=1)
```

What to remember:

- if all vectors are normalized, dot product = cosine similarity
- matrix multiplication gives all-vs-all similarity fast
- greedy is a useful baseline but may fail globally
- when a path problem appears on embeddings, think:
- greedy
- beam search
- shortest path on a graph induced by similarity

11.6 `pdtn2025/Αντίγραφο_knit.ipynb`: optimize line weights to reconstruct an image

Problem pattern:

- output is not labels but parameters of a generative construction
- here the construction is a set of weighted lines
- loss is simply image reconstruction error

What the notebook does:

- defines a differentiable `linesToImage(lines)`
- initializes line weights as zeros
- optimizes those weights directly with SGD
- minimizes mean squared error to the target image
- exports the final weights after scaling/clamping

Important pattern:

```
# `lines` are the actual answer variables, not normal dataset features.
lines = torch.zeros((N//2, N), requires_grad=True)
optimizer = optim.SGD([lines], lr=0.5)

for step in range(num_steps):
    # Render the current candidate image from the current line weights.
    generated_image = linesToImage(lines)

    # Reconstruction loss: the rendered image should match the target image.
    loss = torch.mean((generated_image - target) ** 2)

    # Optimize the line parameters directly.
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

What to remember:

- sometimes the variable you optimize is **not model weights**, but the answer itself
- if the rendering process is differentiable, you can optimize the answer directly
- this is a strong pattern for inverse problems and reconstruction tasks

11.7 `challenges/squarepainting.ipynb`: fit a coordinate -> RGB function with a fixed MLP

Problem pattern:

- each input is a coordinate pair `(x, y)`
- each target is a continuous vector `(r, g, b)`
- architecture is fixed, so tuning happens in the training loop and sampling density

- submission is the trained weights, not a predicted table

What the notebook does:

- samples a dense grid over the unit square
- keeps coordinate inputs and RGB targets as separate tensors
- trains the same fixed architecture on several target functions/images
- renders the learned function back to an image by querying a dense grid
- exports the linear-layer weights to JSON

Why this matters:

- the same pattern appears in coordinate regression, neural fields, image-as-function tasks, and any problem where the answer is a continuous mapping instead of class labels

Reusable dataset pattern:

```
import numpy as np
import torch
from torch.utils.data import TensorDataset, random_split

def generate_ds(target_f, samples=200):
    # Build a dense coordinate grid in [0, 1] x [0, 1].
    steps = np.linspace(0.0, 1.0, samples, dtype=np.float32)
    inputs = []
    targets = []

    for x in steps:
        for y in steps:
            # Each input is one coordinate pair.
            inputs.append([x, y])

            # Each target is the RGB value at that coordinate.
            targets.append(target_f(float(x), float(y)))

    inputs = torch.tensor(inputs, dtype=torch.float32)
    targets = torch.tensor(targets, dtype=torch.float32)
    return TensorDataset(inputs, targets)

full_ds = generate_ds(target_f, samples=200)
train_size = int(0.8 * len(full_ds))
val_size = len(full_ds) - train_size
train_ds, val_ds = random_split(full_ds, [train_size, val_size])
```

Reusable architecture + training loop:

```
import copy
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

def Net():
    # Fixed contest architecture: 2 inputs → hidden MLP → 3 RGB outputs.
    return nn.Sequential(
        nn.Linear(2, 20), nn.ReLU(),
        nn.Linear(20, 20), nn.ReLU(),
        nn.Linear(20, 20), nn.ReLU(),
        nn.Linear(20, 20), nn.ReLU(),
        nn.Linear(20, 20), nn.ReLU(),
        nn.Linear(20, 20), nn.ReLU(),
        nn.Linear(20, 3)
    )

def fit_coordinate_model(target_f, samples=200, epochs=300, batch_size=256):
    device = "cuda" if torch.cuda.is_available() else "cpu"
    model = Net().to(device)

    # Build train/validation splits from one dense coordinate dataset.
    full_ds = generate_ds(target_f, samples=samples)
    train_size = int(0.8 * len(full_ds))
    val_size = len(full_ds) - train_size
    train_ds, val_ds = random_split(full_ds, [train_size, val_size])

    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
num_workers=0)
    val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False, num_workers=0)

    criterion = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr=2e-3)
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.5)

    # Keep the best validation checkpoint, not just the final epoch.
    best_val = float("inf")
    best_state = copy.deepcopy(model.state_dict())

    for epoch in range(epochs):
        model.train()
        for xb, yb in train_loader:
            xb = xb.to(device)
            yb = yb.to(device)

            optimizer.zero_grad()
            pred = model(xb)
            loss = criterion(pred, yb)
            loss.backward()
            optimizer.step()

        model.eval()
        total_val_loss = 0.0
        with torch.no_grad():
            for xb, yb in val_loader:
                xb = xb.to(device)
                yb = yb.to(device)
```

```
                total_val_loss += criterion(model(xb), yb).item()

        val_loss = total_val_loss / len(val_loader)
        scheduler.step()

        if val_loss < best_val:
            best_val = val_loss
            best_state = copy.deepcopy(model.state_dict())

    model.load_state_dict(best_state)
    return model
```

Reusable export pattern:

```
def export_weights(net):
    layers = []

    for layer in net:
        if isinstance(layer, nn.Linear):
            # Export only trainable linear layers in plain Python lists.
            layers.append({
                "weights": layer.weight.detach().cpu().tolist(),
                "bias": layer.bias.detach().cpu().tolist(),
            })

    return {"layers": layers}
```

Important failure modes:

- do **not** `stack` `(N, 2)` coordinate inputs with `(N, 3)` RGB targets
- keep targets as `float32`
- if the fit is blocky, increase sampling density before changing everything else
- if validation is much worse than train, keep a best checkpoint and reduce LR
- on Windows/notebooks, set `num_workers=0` first if loader workers crash

What to remember:

- this is supervised regression, not classification
- the final layer should output `3` floats, one for each RGB channel
- `MSELoss` is the natural first loss
- the answer can be the model weights themselves when the contest wants a function representation

12. Highest-Value Local Repo Files

Review these before the contest:

- `comp/train_func.py`
- `train_function.py`
- `challenges/mnist1.py`
- `challenges/cfar1.py`
- `challenges/CFAR1001.py`
- `challenges/nlp.py`
- `challenges/extracted.py`
- `decision_trees.py`

- `comp/training_bug.md`
- `comp/tutor_advice_classification_vs_regression.md`
- `emotions.ipynb`
- `challenges/nextmovie.ipynb`
- `challenges/captcha.ipynb`
- `challenges/squarepainting.ipynb`
- `challenges/squarepainting.py`
- `pdtn2025/deepfakes_final.ipynb`
- `pdtn2025/Αντίγραφο_embeddings_greedy_torch (1).ipynb`
- `pdtn2025/Αντίγραφο_knit.ipynb`

13. Final Reminder

The usual winning pattern is:

- identify the task correctly
- choose the correct cheap baseline
- validate quickly
- submit
- improve only where evidence says it is worth it